WEB3 EXPERTS BRAZIL

Formal Verification for Web3

Christiano Braga

ZKForall

Universidade Federal Fluminense

Semar Martins

ZKForall

June 27, 2026

About This Workshop

Duration: 90 minutes

Part 1 — Lean 4: Foundations and Tactics (~45 min)

- Propositions, types, tactics
- Interactive exercises

Part 2 — Cryptographic Protocols in Lean 4 (~45 min)

- Deterministic algebraic model
- One-Time Pad, RSA, Diffie-Hellman
- Interactive exercises

Motivation

The AI Revolution in Software Development

AI Is Writing Our Code

- GitHub Copilot: **46% of code** is now AI-generated, up from 27% at launch in 2022
- **15 million developers** using Copilot — 400% growth in one year
- AI agents writing entire modules, tests, deployments
- Smart contracts worth billions deployed with AI assistance

> "Move fast and break things" is not acceptable when the thing is a $600M bridge contract.

GitHub, "Octoverse 2024." See also GitClear, "Coding on Copilot," 2024.

The Fundamental Problem

AI is powerful but not trustworthy by default.

- LLMs hallucinate — confidently and fluently
- Code "looks right" but may be subtly wrong
- Testing catches some bugs; formal verification catches **all** bugs in scope

Who verifies the AI?

AI Meets Web3

- **AI-assisted smart contract development:** Copilot, ChatGPT, and Claude generate Solidity, Move, and Rust contracts daily
- **AI-powered auditing:** tools like Olympix and Cyfrin use AI to scan for vulnerabilities
- **The paradox:** AI finds bugs *and* introduces bugs

Real-world impact:

- Wormhole bridge hack ($320M, 2022) — bypassed signature verification
- Euler Finance ($197M, 2023) — missing health check in donation function
- Could AI have caught these? Could AI have *caused* them?

> Formal verification is the missing layer: it does not matter who writes the code — human or AI — the proof must be machine-checked.

Halborn, "Explained: The Wormhole Hack," 2022.
Cyfrin, "Euler Finance Hack Analysis," 2023.

Why Formal Verification for Web3?

Property	Testing	Formal Verification
Coverage	Sampled inputs	**All** possible inputs
Guarantees	Probabilistic	**Mathematical proof**
Smart contract bugs	Catches some	Proves absence
Cost of failure	High	**Prevented**

Web3 is adversarial by design. Every smart contract is an open invitation to exploit.

Runtime Verification, "Formal Verification 101 for Blockchain Systems and Smart Contracts," 2020.

Why Lean 4?

Lean 4 uniquely combines **programming** and **proving** in one language — code and its correctness proof live side by side, checked by the same compiler.

- **Dependent types** — types can depend on values, so invariants (e.g., "this key has 256 bits") are enforced at compile time
- **Mathlib** — 300k+ formally verified theorems, including the number theory and algebra our proofs need
- **General-purpose** — compiles to C with production-ready performance
- **AI integration** — LLMs are already being trained to generate Lean proofs (e.g., Google DeepMind's AlphaProof). Lean is both the language AI writes proofs in *and* the tool that checks them

> AI proposes, Lean verifies.

Lean: https://lean-lang.org/ — Mathlib: https://leanprover-community.github.io/

The mathlib Community, "The Lean Mathematical Library," CPP 2020.

Part 1

Lean 4: Foundations and Tactics

Logic Refresher: Connectives

A **proposition** is a statement that is either true or false. We build complex propositions from simpler ones:

Symbol	Name	Read as	True when
$P \wedge Q$	Conjunction	"P and Q"	Both P and Q hold
$P \vee Q$	Disjunction	"P or Q"	At least one holds
$P \to Q$	Implication	"P implies Q"	If P then Q
$\neg P$	Negation	"not P"	P does not hold

Modus ponens: from P and $P \to Q$, conclude Q.
Proof by cases: to prove $P \vee Q \to R$, prove $P \to R$ and $Q \to R$ separately.

Logic Refresher: Quantifiers and Equality

Quantifiers let us talk about *all* or *some* elements of a type:

Symbol	Name	Read as	Example
$\forall x, P(x)$	Universal	"for all x, P(x)"	$\forall n \in \mathbb{N},\ n + 0 = n$
$\exists x, P(x)$	Existential	"there exists x such that P(x)"	$\exists n \in \mathbb{N},\ n > 5$

Equality ($=$) is the most common relation:

- **Reflexivity:** $a = a$
- **Symmetry:** if $a = b$ then $b = a$
- **Transitivity:** if $a = b$ and $b = c$ then $a = c$
- **Substitution:** if $a = b$ then $f(a) = f(b)$

These are exactly the reasoning principles that Lean's tactics automate.

What Is Lean 4?

A **dependently-typed** functional programming language that doubles as an interactive theorem prover.

```
-- A program
def double (n : Nat) : Nat := 2 * n

-- A theorem
theorem double_is_add_self (n : Nat) : double n = n + n := by
  rewrite [double]
  -- ⊢ 2 * n = n + n
  rewrite [Nat.succ_mul]
  -- ⊢ 1 * n + n = n + n
  rewrite [Nat.one_mul]
  -- ⊢ n + n = n + n
  rfl
```

Key idea: **propositions are types, proofs are programs** (Curry-Howard correspondence).

Try in Lean 4 Web

Anatomy of a Lean Proof

```
-- The prime (') avoids clashing with Mathlib names
theorem succ_pos' (n : Nat) : 0 < n + 1 := by
--       ^name     ^params   ^statement      ^tactic block
  exact Nat.succ_pos n
--^tactic (one step of the proof)
```

Keyword	Meaning
`def`	Define a function or value
`theorem` / `lemma`	State and prove a proposition
`: T`	Type annotation — "this has type T"
`:= by`	"prove this using tactics" (enters tactic mode)
`sorry`	Placeholder — admits the goal without proof (marks it with a warning)
`#check` / `#eval`	Inspect a type / evaluate an expression

Propositions as Types

Logic	Lean Type
$P \wedge Q$	`P × Q` (product)
$P \vee Q$	`P ⊕ Q` (sum)
$P \to Q$	`P → Q` (function)
$\forall x, P(x)$	`(x : α) → P x` (dependent function)
$\exists x, P(x)$	`⟨x, proof⟩` (Sigma type)
True	`True` (unit)
False	`False` (empty)

Howard, W., "The formulae-as-types notion of construction," 1969/1980. See also: de Bruijn, N., "AUTOMATH," 1968.

Basic Types and Terms

```
#check (42 : Nat)                -- a natural number
#check Nat.succ 3                -- 4 (successor)
def add_one (n : Nat) : Nat := n + 1
#eval add_one 41                 -- 42
#check (2 + 2 = 4 : Prop)      -- a proposition
#check (∀ n : Nat, n + 0 = n) -- also a proposition
```

Types and propositions live in the same world: `Nat` is a type of data, `2 + 2 = 4` is a type of evidence (a `Prop`). A `theorem` is a function that produces evidence.

Try in Lean 4 Web

Structures and Notation

A `structure` bundles named fields (used heavily in Part 2 for crypto parameters):

```
structure Point where      -- like a record / named tuple
  x : Nat
  y : Nat

abbrev MyType := Nat       -- type alias (abbrev is transparent)
```

Key notation used throughout this workshop:

Syntax	Meaning
`⟨a, b⟩`	Anonymous constructor — build a struct or existential witness
`h.1` / `h.2`	First / second projection (e.g., left / right of `∧`)
`x.field`	Structure field access: `params.e`, `m.val`
`· + ·`	Anonymous function: `fun a b => a + b`
`tac1; tac2`	Tactic chaining — run `tac1` then `tac2` on same line

What Are Tactics?

A **tactic** is a command that transforms the current proof goal into simpler subgoals, until none remain.

```
theorem example_proof (P Q : Prop) (hp : P) (hpq : P → Q) : Q := by
  -- Goal: Q
  exact hpq hp
  -- No goals remain ✓
```

Why tactics? Writing proof terms directly (like `hpq hp`) is possible but quickly becomes unreadable for complex proofs. Tactics let us build proofs **step by step**, the way a mathematician would reason on a whiteboard.

For example, `fun h => ⟨h.2, h.1⟩` is a valid term-mode proof, but the tactic version `intro h; exact ⟨h.2, h.1⟩` lets us inspect the goal at each step.

Tactic: `rfl`

When? Verifying that two expressions compute to the same value — e.g., checking a hash function output or a contract's arithmetic.

`rfl` proves goals of the form `a = a` after Lean reduces both sides.

```
theorem two_plus_two : 2 + 2 = 4 := by
  rfl

theorem bool_and_true : true && true = true := by
  rfl
```

Lean reduces `2+2` to `4`, then sees `4 = 4`.

Try in Lean 4 Web

Tactic: `exact` and `assumption`

When? You have exactly the proof you need in the context. Like citing a known fact to close an argument.

```
-- exact: provide the precise proof term
theorem and_intro' (P Q : Prop) (hp : P) (hq : Q) : P ∧ Q := by
  exact ⟨hp, hq⟩

-- assumption: let Lean search the context for you
theorem trivial_imp (P : Prop) (h : P) : P := by
  assumption
```

Try in Lean 4 Web

Tactic: `intro`

When? Your goal starts with $\forall$ or $\to$. Like saying "let `n` be an arbitrary natural number" or "assume `P` holds."

```
theorem identity_eq : ∀ (n : Nat), n = n := by
  intro n   -- now n : Nat is in context, goal is n = n
  rfl

theorem imp_self' (P : Prop) : P → P := by
  intro h   -- assume P holds as hypothesis h
  exact h
```

Try in Lean 4 Web

Tactic: `simp`

When? The goal involves known simplification rules — like `x + 0 = x`, `l ++ [] = l`. The workhorse tactic that automates routine rewriting.

```
theorem list_append_nil (l : List α) : l ++ [] = l := by
  simp

theorem arith_example (n : Nat) : n + 0 + 0 = n := by
  simp
```

You can supply extra lemmas: `simp [my_lemma]`.

Variant `simp only [lemmas]` — rewrite using **only** the listed lemmas (no default simp set). Predictable but more verbose; preferred when you want to control which rewrites fire (e.g., `simp only [dh_shared_secret]` just unfolds the definition).

Try in Lean 4 Web

Tactic: `decide`

When? The goal is a proposition over a **finite** type that Lean can check by trying all cases. Lean enumerates every possibility and verifies the statement holds for each one.

```
-- Fin 2 has only two elements: 0 and 1
-- Lean checks both: 0+0=0 ✓, 1+1=0 ✓
theorem fin2_self_add : ∀ (a : Fin 2), a + a = 0 := by
  decide

-- Works for any decidable proposition over finite types
theorem bool_demorgan : ∀ (a b : Bool),
    (!(a && b)) = (!a || !b) := by
  decide   -- checks all 4 combinations of true/false
```

Note: `decide` requires **closed** propositions (no free variables). Use `∀` to quantify.

Try in Lean 4 Web

Tactic: `omega`

When? The goal is a linear arithmetic statement over `Nat` or `Int` — e.g., proving bounds, fee calculations, or index validity.

```
theorem omega_demo1 (n : Nat) : n + 1 > n := by
  omega

theorem omega_demo2 (a b : Nat) (h : a ≤ b) :
    a + 1 ≤ b + 1 := by
  omega
```

Decides all true linear arithmetic goals automatically.

Try in Lean 4 Web

Tactic: `ring`

When? Proving polynomial or algebraic equalities — e.g., verifying that a fee formula distributes correctly or that a cryptographic identity holds.

```
import Mathlib.Tactic.Ring  -- ring requires Mathlib

theorem ring_demo1 (a b : Nat) :
    (a + b) * (a + b) = a*a + 2*a*b + b*b := by
  ring

theorem ring_demo2 (x : Int) :
    (x + 1) * (x - 1) = x^2 - 1 := by
  ring
```

Variant `ring_nf` — *normalizes* the goal to a canonical polynomial form without insisting it be a closed equality. Use it as a step in a longer proof, e.g. `rw [← pow_mul, ← pow_mul]; ring_nf` to massage exponents into the same shape.

Try in Lean 4 Web

Tactic: `have` and `calc`

When? A proof needs intermediate steps — like showing `a = b` then `b = c` to conclude `a = c`. Essential for readable, auditable proofs.

```
theorem have_demo (a b c : Nat)
    (h1 : a = b) (h2 : b = c) : a = c := by
  have h3 : a = c := by
    calc a = b := h1
         _ = c := h2
  exact h3
```

`calc` chains are great for step-by-step equational reasoning.

Try in Lean 4 Web

Tactics: `unfold`, `rw`, `ext`

These appear frequently in Part 2's cryptographic proofs:

Tactic	Purpose	Example use
`unfold f`	Expand definition `f` in the goal	`unfold otp_encrypt`
`rewrite [h]`	Rewrite using `h : a = b`; **leaves** the resulting goal open	step-by-step proofs
`rw [h]`	Same as `rewrite`, then tries `rfl` to close the goal	`rw [pow_mul]`
`rw [← h]`	Rewrite in reverse direction	`rw [← add_assoc]`
`ext i`	Prove equality component-wise	Two `Vector`s equal element-wise

```
def double' (n : Nat) : Nat := 2 * n

theorem double_eq (n : Nat) : double' n = 2 * n := by
  unfold double'  -- goal becomes 2 * n = 2 * n

theorem rw_demo (a b : Nat) (h : a = b) : a + 1 = b + 1 := by
  rw [h]          -- replaces a with b
```

Tactic Summary

Tactic	Purpose	Used in
`rfl`	Prove `a = a` (definitional equality)	Warm-up
`intro` / `exact`	Move quantifiers into context / provide proof term	Warm-up
`simp`	Simplification with a lemma database	OTP, DH
`decide`	Brute-force decidable propositions	OTP
`omega`	Decide linear arithmetic ($\mathbb{N}$, $\mathbb{Z}$)	RSA
`ring` / `ring_nf`	Prove ring equalities / normalize	OTP, RSA, DH
`have` / `calc`	Introduce intermediate steps	OTP
`unfold`	Expand a definition in the goal	OTP, RSA
`rw` / `rw [←]`	Rewrite with an equality	RSA, DH
`ext`	Prove equality component-wise	OTP

Exercise 1: Logic Warmup

```
-- 1a. Commutativity of ∧
theorem and_comm_ex (P Q : Prop) : P ∧ Q → Q ∧ P := by
  sorry

-- 1b. Transitivity of implication
theorem imp_trans' (P Q R : Prop) :
    (P → Q) → (Q → R) → P → R := by
  sorry

-- 1c. A simple arithmetic fact
theorem exercise_arith (n : Nat) : n + n = 2 * n := by
  sorry
```

Hint: For 1a, use `intro h` then `exact ⟨h.2, h.1⟩`

Try in Lean 4 Web

Solution 1

```lean
-- 1a
theorem and_comm_ex (P Q : Prop) : P ∧ Q → Q ∧ P := by
  intro h
  exact ⟨h.2, h.1⟩

-- 1b
theorem imp_trans' (P Q R : Prop) :
    (P → Q) → (Q → R) → P → R := by
  intro hpq hqr hp
  exact hqr (hpq hp)

-- 1c
theorem exercise_arith (n : Nat) : n + n = 2 * n := by
  omega
```

Try in Lean 4 Web

Part 2

Cryptographic Protocols in Lean 4

Mathematics refresher

Modular Arithmetic

Integers modulo n ($\mathbb{Z}_n$ or `ZMod n` in Lean/Mathlib): the set $\{0, 1, \ldots, n-1\}$ with wrap-around arithmetic.

Operation	Definition	Example ($\mathbb{Z}_5$)
$a + b \bmod n$	Addition mod n	$3 + 4 = 2$
$a \cdot b \bmod n$	Multiplication mod n	$3 \cdot 4 = 2$
$a^k \bmod n$	Exponentiation mod n	$2^3 = 3$

Key special case — $\mathbb{Z}_2$:

- Only two elements: 0 and 1
- Addition mod 2 **is** XOR: $1 + 1 = 0, 0 + 1 = 1$
- Used in One-Time Pad

Number Theory for Cryptography

Euler's totient $\phi(n)$: count of integers in $\{1, \ldots, n\}$ coprime to n.

- For primes: $\phi(p) = p - 1$
- For $n = p \cdot q$ (distinct primes): $\phi(n) = (p-1)(q-1)$

Euler's theorem: If $\gcd(m, n) = 1$, then $m^{\phi(n)} \equiv 1 \pmod{n}$

Consequence for RSA: If $e \cdot d \equiv 1 \pmod{\phi(n)}$, then:

$$m^{e \cdot d} = m^{1+k\phi(n)} = m \cdot (m^{\phi(n)})^k \equiv m \pmod{n}$$

Commutativity of exponents: $(g^a)^b = g^{ab} = g^{ba} = (g^b)^a$ — the basis of Diffie-Hellman.

The Deterministic Cryptographic Model

We use a **deterministic algebraic model** of cryptography:

- No probability distributions or computational complexity
- Security properties are stated as **algebraic equalities**
- Focus on **correctness**: decryption inverts encryption
- Focus on **structural properties**: commutativity, key independence

Not modeled: computational hardness, random oracles, adversarial games.

For computational models see: Blanchet, B., "A Computationally Sound Mechanized Prover for Security Protocols," IEEE TDSC, 2008; Barthe, G. et al., "Computer-Aided Security Proofs for the Working Cryptographer," CRYPTO, 2011.

Protocol 1: One-Time Pad (XOR Cipher)

Idea: Encrypt by XOR-ing plaintext with a key of equal length.

$$c = m \oplus k \qquad\qquad m = c \oplus k$$

Perfect secrecy: With a truly random key used once, the ciphertext reveals nothing about the plaintext.

What we'll prove:

1. Correctness: decryption recovers the plaintext
2. XOR algebraic properties (involution, commutativity)

Shannon, C.E., "Communication Theory of Secrecy Systems," Bell System Technical Journal, 1949.

OTP in Lean 4: Definitions

```
import Mathlib.Data.ZMod.Basic
import Mathlib.Data.Vector.Basic

-- We model bits as ZMod 2 (integers mod 2)
-- XOR is simply addition in ZMod 2
abbrev Bit := ZMod 2

def otp_encrypt (key msg : Vector Bit n) : Vector Bit n :=
  Vector.zipWith (· + ·) key msg

def otp_decrypt (key cipher : Vector Bit n) : Vector Bit n :=
  Vector.zipWith (· + ·) key cipher
```

Try in Lean 4 Web

OTP: Correctness Proof

```
-- Key property of ZMod 2: a + a = 0
theorem zmod2_self_add : ∀ (a : ZMod 2), a + a = 0 := by
  decide

-- Correctness: decrypt(key, encrypt(key, msg)) = msg
theorem otp_correct (key msg : Vector Bit n) :
    otp_decrypt key (otp_encrypt key msg) = msg := by
  unfold otp_encrypt otp_decrypt
  ext i
  simp [Vector.zipWith]
  -- Goal: key[i] + (key[i] + msg[i]) = msg[i]
  -- By associativity: (key[i] + key[i]) + msg[i]
  -- By self-inverse:  0 + msg[i] = msg[i]
  rw [← add_assoc, zmod2_self_add, zero_add]
```

Try in Lean 4 Web

OTP: Key Properties

```
-- Encryption is its own inverse (involutory)
theorem otp_involutory (key msg : Vector Bit n) :
    otp_encrypt key (otp_encrypt key msg) = msg :=
  otp_correct key msg  -- encrypt and decrypt are the same!

-- Encryption is commutative in key and message
theorem otp_comm (a b : Vector Bit n) :
    otp_encrypt a b = otp_encrypt b a := by
  unfold otp_encrypt; ext i
  simp [Vector.zipWith]; ring
```

Try in Lean 4 Web

Exercise 3: OTP Properties

```
-- 3a. Encrypting with the zero key is identity
theorem otp_zero_key (msg : Vector Bit n) :
    otp_encrypt (Vector.replicate n (0 : Bit)) msg
    = msg := by
  sorry

-- 3b. Encrypting a message with itself yields zeros
theorem otp_self_encrypt (msg : Vector Bit n) :
    otp_encrypt msg msg
    = Vector.replicate n (0 : Bit) := by
  sorry
```

Hint: `unfold`, `ext i`, `simp`, and `zmod2_self_add`.

Try in Lean 4 Web

Solution 3

```
-- 3a
theorem otp_zero_key (msg : Vector Bit n) :
    otp_encrypt (Vector.replicate n (0 : Bit)) msg
    = msg := by
  unfold otp_encrypt; ext i
  simp [Vector.zipWith, Vector.replicate]

-- 3b
theorem otp_self_encrypt (msg : Vector Bit n) :
    otp_encrypt msg msg
    = Vector.replicate n (0 : Bit) := by
  unfold otp_encrypt; ext i
  simp [Vector.zipWith]
  exact zmod2_self_add (msg.get i)
```

Try in Lean 4 Web

Protocol 2: RSA

Setup:

- Choose primes p, q; compute $n = p \cdot q$
- Choose e **coprime** to $\phi(n) = (p-1)(q-1)$ — i.e., $\gcd(e, \phi(n)) = 1$, so that e has a multiplicative inverse mod $\phi(n)$
- Compute $d = e^{-1} \mod \phi(n)$

Operations:

$$\text{Encrypt: } c = m^e \bmod n \qquad \text{Decrypt: } m = c^d \bmod n$$

Correctness relies on Euler's theorem: $m^{e \cdot d} \equiv m \pmod{n}$

Rivest, R., Shamir, A., Adleman, L., "A Method for Obtaining Digital Signatures and Public-Key Cryptosystems," CACM, 1978.

RSA in Lean 4: Definitions

```
import Mathlib.Data.ZMod.Basic; import Mathlib.Data.Nat.Prime.Basic

structure RSAParams where
  p : Nat;  q : Nat
  hp : Nat.Prime p;  hq : Nat.Prime q;  hpq : p ≠ q
  e : Nat;  d : Nat;  he_pos : 0 < e
  n : Nat := p * q;  phi : Nat := (p - 1) * (q - 1)
  he : Nat.Coprime e phi;  hed : e * d % phi = 1

def rsa_encrypt (params : RSAParams) (m : ZMod params.n) :=
  m ^ params.e
def rsa_decrypt (params : RSAParams) (c : ZMod params.n) :=
  c ^ params.d
```

Try in Lean 4 Web

RSA: Correctness Statement

```
theorem rsa_correct (params : RSAParams)
    (m : ZMod params.n)
    (hm : Nat.Coprime m.val params.n) :
    rsa_decrypt params (rsa_encrypt params m) = m := by
  unfold rsa_encrypt rsa_decrypt
  rw [← pow_mul]
  -- Goal: m ^ (e * d) = m  in ZMod n
  -- e * d ≡ 1 (mod φ(n)) → e * d = 1 + k * φ(n)
  -- By Euler: m ^ φ(n) ≡ 1 (mod n)
  -- So: m ^ (1 + k * φ(n)) = m * 1^k = m
  sorry -- requires Euler's theorem from Mathlib
```

The `sorry` is intentional — completing this proof requires Euler's theorem, which is available in Mathlib as `ZMod.pow_totient`.

RSA: Why `sorry` and What Is CRT?

Euler's theorem states that $m^{\phi(n)} \equiv 1 \pmod{n}$ when $\gcd(m, n) = 1$. In Mathlib, this is `ZMod.pow_totient`: for coprime m and n, raising m to Euler's totient $\phi(n)$ gives 1 in `ZMod n`. Chaining this with $e \cdot d \equiv 1 \pmod{\phi(n)}$ proves $m^{ed} = m$.

The `hm` hypothesis requires $\gcd(m, n) = 1$ (i.e., the message is coprime to n). This excludes messages that are multiples of p or q — extremely unlikely in practice since p and q are large primes.

CRT (Chinese Remainder Theorem): For $n = p \cdot q$, CRT gives an isomorphism $\mathbb{Z}_n \cong \mathbb{Z}_p \times \mathbb{Z}_q$. This allows proving $m^{ed} = m$ for **all** m (including non-coprime ones) by checking the identity modulo p and modulo q separately. This is a longer proof that we omit here.

RSA: Algebraic Properties

```
-- RSA is homomorphic: E(m1) * E(m2) = E(m1 * m2)
theorem rsa_homomorphic (params : RSAParams)
    (m1 m2 : ZMod params.n) :
    rsa_encrypt params m1 * rsa_encrypt params m2
    = rsa_encrypt params (m1 * m2) := by
  unfold rsa_encrypt; rw [← mul_pow]

-- Encrypt and decrypt commute
theorem rsa_commute (params : RSAParams)
    (m : ZMod params.n) :
    (m ^ params.e) ^ params.d
    = (m ^ params.d) ^ params.e := by
  rw [← pow_mul, ← pow_mul]; ring_nf
```

Try in Lean 4 Web

Exercise 4: RSA Properties

```
-- 4a. Encrypting 1 always gives 1
theorem rsa_one (params : RSAParams) :
    rsa_encrypt params (1 : ZMod params.n) = 1 := by
  sorry

-- 4b. Encrypting 0 always gives 0
theorem rsa_zero (params : RSAParams) :
    rsa_encrypt params (0 : ZMod params.n) = 0 := by
  sorry
```

Hint: Unfold `rsa_encrypt` and use `one_pow` / `zero_pow`.

Try in Lean 4 Web

Solution 4

```
-- 4a
theorem rsa_one (params : RSAParams) :
    rsa_encrypt params (1 : ZMod params.n) = 1 := by
  unfold rsa_encrypt
  exact one_pow params.e

-- 4b
theorem rsa_zero (params : RSAParams) :
    rsa_encrypt params (0 : ZMod params.n) = 0 := by
  unfold rsa_encrypt
  exact zero_pow params.he_pos.ne'
```

Try in Lean 4 Web

Protocol 3: Diffie-Hellman Key Exchange

Setup: Public prime p, generator g of $\mathbb{Z}_p^*$

1. Alice chooses secret a, sends $A = g^a \bmod p$
2. Bob chooses secret b, sends $B = g^b \bmod p$
3. **Shared secret:** $s = B^a = A^b = g^{ab} \bmod p$

Key property: Both parties compute the same shared secret.

Diffie, W. and Hellman, M., "New Directions in Cryptography," IEEE Trans. Info. Theory, 1976.

DH in Lean 4: Definitions

```
import Mathlib.Data.ZMod.Basic

structure DHParams where
  p : Nat
  hp : Nat.Prime p
  g : ZMod p          -- generator (in ZMod p, not Z_p^*)

structure DHKeyPair (params : DHParams) where
  secret : Nat
  public_key : ZMod params.p := params.g ^ secret

def dh_shared_secret (params : DHParams)
    (my_secret : Nat) (their_public : ZMod params.p)
    : ZMod params.p :=
  their_public ^ my_secret
```

Try in Lean 4 Web

DH: Correctness Proof

```
-- Both sides compute the same shared secret
theorem dh_shared_secret_agree (params : DHParams)
    (a b : Nat) :
    let alice_pub := params.g ^ a
    let bob_pub   := params.g ^ b
    dh_shared_secret params a bob_pub =
    dh_shared_secret params b alice_pub := by
  simp only [dh_shared_secret]
  -- Goal: (g ^ b) ^ a = (g ^ a) ^ b
  rw [← pow_mul, ← pow_mul]
  -- Goal: g ^ (b * a) = g ^ (a * b)
  ring_nf
```

The proof hinges on a single algebraic fact: multiplication of natural numbers is commutative ($b \cdot a = a \cdot b$), so $g^{ba} = g^{ab}$. This is why both parties end up with the same shared secret.

DH: A Note on Our Model

We work in `ZMod p` — the ring of **all** residues modulo p — rather than $\mathbb{Z}_p^*$, the multiplicative group of units (non-zero elements with multiplicative inverses).

In the real Diffie-Hellman protocol, g must be a **generator** of $\mathbb{Z}_p^*$, meaning every non-zero element can be written as g^k for some k. This is essential for security: if g generates only a small subgroup, an attacker can brute-force the discrete logarithm.

However, the exponent identity $(g^b)^a = g^{ba}$ holds in **any monoid** (a set with an associative operation and identity element). Since `ZMod p` under multiplication is a monoid, our algebraic correctness proof is valid regardless of the choice of g.

DH: Additional Properties

```
-- Extends to 3-party key agreement
theorem dh_three_party (params : DHParams) (a b c : Nat) :
    ((params.g ^ a) ^ b) ^ c
    = ((params.g ^ a) ^ c) ^ b := by
  rw [← pow_mul, ← pow_mul, ← pow_mul, ← pow_mul]; ring_nf

-- Secret 0 gives trivial key (g^0 = 1)
theorem dh_zero_secret (params : DHParams)
    (their_pub : ZMod params.p) :
    dh_shared_secret params 0 their_pub = 1 := by
  simp [dh_shared_secret]
```

Try in Lean 4 Web

Exercise 5: Diffie-Hellman

```
-- 5a. Shared secret with yourself: g^(a*a)
theorem dh_self (params : DHParams) (a : Nat) :
    dh_shared_secret params a (params.g ^ a)
    = params.g ^ (a * a) := by
  sorry

-- 5b. Composing exponentiations
theorem dh_compose (params : DHParams) (a b : Nat)
    (their_pub : ZMod params.p) :
    (their_pub ^ a) ^ b = their_pub ^ (a * b) := by
  sorry
```

Hint: Use `pow_mul` and `ring_nf`.

Try in Lean 4 Web

Solution 5

```
-- 5a
theorem dh_self (params : DHParams) (a : Nat) :
    dh_shared_secret params a (params.g ^ a)
    = params.g ^ (a * a) := by
  simp [dh_shared_secret]
  rw [← pow_mul]

-- 5b
theorem dh_compose (params : DHParams) (a b : Nat)
    (their_pub : ZMod params.p) :
    (their_pub ^ a) ^ b = their_pub ^ (a * b) := by
  rw [← pow_mul]
```

Try in Lean 4 Web

Bonus: ? operator

```
theorem dh_self (params : DHParams) (a : Nat) :
    dh_shared_secret params a (params.g ^ a)
    = params.g ^ (a * a) := by
  simp only [dh_shared_secret]
  -- try this: rw?
  rw [pow_mul]

theorem dh_compose (params : DHParams) (a b : Nat)
    (their_pub : ZMod params.p) :
    (their_pub ^ a) ^ b = their_pub ^ (a * b) := by
      /- try this: apply? -/
      exact Eq.symm (pow_mul their_pub a b)
```

Try in Lean 4 Web

Comparing the Three Protocols

	OTP	RSA	Diffie-Hellman
Type	Symmetric	Asymmetric	Key Exchange
Key idea	XOR involution	Euler's theorem	Commutativity
Proof core	$a + a = 0$ in $\mathbb{Z}_2$	$m^{ed} = m$ in $\mathbb{Z}_n$	$g^{ab} = g^{ba}$
Tactic	`ring`, `ext`	`pow_mul`	`pow_mul`, `ring_nf`

Beyond This Workshop

Our **deterministic model** is a deliberate starting point — it proves algebraic correctness cleanly and accessibly. A complete cryptographic treatment would also address:

- **Computational hardness** — factoring, discrete log assumptions
- **Probabilistic security** — IND-CPA, IND-CCA game-based definitions
- **Side channels** — timing, power analysis
- **Key management** — generation, distribution, revocation

These are active areas of formal verification research:

- CryptoVerif (Blanchet, 2008) — computational model
- EasyCrypt (Barthe et al., 2011) — game-based proofs
- **Lean + Mathlib** — growing support for probability theory

Where This Workshop Fits in Web3

```
┌──────────────────────────────────────┐
│     Smart Contract (Solidity/Move)   │
├──────────────────────────────────────┤
│     Formal Spec (Lean 4)             │   ← Part 1: language & tactics
├──────────────────────────────────────┤
│     Cryptographic Primitives         │   ← Part 2: OTP, RSA, DH
├──────────────────────────────────────┤
│     Consensus Protocol               │
├──────────────────────────────────────┤
│     Network Layer                    │
└──────────────────────────────────────┘
```

In Part 1 we learned the **verification language** (Lean 4 foundations and tactics). In Part 2 we applied it to the **cryptographic primitives layer**, proving correctness of OTP, RSA, and Diffie-Hellman. The same techniques scale upward to verify smart contracts and consensus protocols.

Lean Verification in Web3 Today

Projects already using Lean and formal methods for Web3 artifacts:

- **Nethermind CertiPlonk** — extracts and formally verifies ZK circuit constraints from the Plonky3 zkDSL in Lean
- **Nethermind Halo2 Verification** — formal verification of Halo2 ZK circuits in Lean
- **INTMAX2 Financial Safety** — Nethermind verified the financial safety properties of the INTMAX2 stateless zkRollup in Lean
- **CertiK zkWasm** — first full formal verification of a zkVM (33k lines of Coq)
- **ZKProof Verified Verifiers** — community effort to formally verify ZK proof system verifiers

Nethermind, "Formally Verifying Zero-Knowledge Circuits," 2025.

Key Takeaways

1. **AI writes code; Lean verifies it.** The prover doesn't care who wrote the code — it only accepts mathematical proof.
2. **Cryptographic correctness is provable.** Even simple algebraic models catch real bugs in protocol logic.
3. **Lean 4 is practical.** A real programming language with a real type checker — not just academic.
4. **Web3 needs this.** Immutable contracts with billions at stake demand mathematical guarantees.
5. **Start small.** The exercises in this workshop are real Lean 4 — install it and keep practicing.

References

- Dalrymple, D. et al., "Towards Guaranteed Safe AI," arXiv:2405.06624, 2024.
- Tegmark, M. and Omohundro, S., "Provably safe systems," arXiv:2309.01933, 2023.
- De Moura, L. and Ullrich, S., "The Lean 4 Theorem Prover," CADE-28, 2021.
- Bursuc, S. et al., "A benchmark for vericoding," arXiv:2509.22908, 2025.
- Shannon, C.E., "Communication Theory of Secrecy Systems," Bell Syst. Tech. J., 1949.
- Rivest, R., Shamir, A., Adleman, L., "Digital Signatures and Public-Key Cryptosystems," CACM, 1978.
- Diffie, W. and Hellman, M., "New Directions in Cryptography," IEEE Trans. IT, 1976.
- Blanchet, B., "Computationally Sound Mechanized Prover," IEEE TDSC, 2008.
- Barthe, G. et al., "Computer-Aided Security Proofs," CRYPTO, 2011.
- The mathlib Community, "The Lean Mathematical Library," CPP, 2020.

Resources

- **Lean 4**: https://lean-lang.org/
- **Lean 4 Web IDE**: https://live.lean-lang.org/
- **Mathlib**: https://leanprover-community.github.io/
- **Theorem Proving in Lean 4**: https://lean-lang.org/theorem_proving_in_lean4/
- **Mathematics in Lean**: https://leanprover-community.github.io/mathematics_in_lean/
- **BAIF Research**: https://www.beneficialaifoundation.org/research

Thank You

Questions?

Web3 Experts Brazil — March 2026

"In mathematics we trust — but only if it's machine-checked."

Appendix: Setting Up Lean 4

```
# Install elan (Lean version manager)
curl https://elan.lean-lang.org/install.sh -sSf | sh

# Create a Mathlib-enabled project
lake new my_crypto_proofs math
cd my_crypto_proofs

# Download precompiled Mathlib (otherwise lake build
# recompiles Mathlib from source — hours)
lake exe cache get
lake build
```

The `math` template wires up every Mathlib module the slides import.

No local install? Each code block has a *Try in Lean 4 Web* link — https://live.lean-lang.org/. **Editor:** VS Code with the `lean4` extension.